

高速车流量计数器实战实验报告

1. 实验目的

本实验围绕高速车流量计数任务，完成基于计算机视觉的视频目标检测、目标追踪、过线计数和前端可视化。实验目标包括：搭建可运行的深度学习环境，运行 YOLOv5 + DeepSORT pipeline，对测试视频和课后作业视频进行检测计数，完成 VOC XML 到 YOLO txt 的数据格式转换，启动模型训练并生成权重文件，最后使用 Gradio 前端展示检测与计数效果。

- 理解 YOLOv5 目标检测模型在车辆、人等交通目标识别中的作用。
- 掌握 DeepSORT 根据目标外观特征和运动状态进行 ID 追踪的基本流程。
- 掌握视频逐帧读取、目标框绘制、计数线绘制和结果视频保存方法。
- 完成数据标注、格式转换、训练、推理和前端可视化的完整项目链路。

2. 实验环境与材料

类别	配置或文件
硬件	NVIDIA GeForce RTX 4060 Laptop GPU，显存约 8GB
Python 环境	E:\anaconda3\envs\torch\python.exe; Python 3.12.13
深度学习框架	torch 2.11.0+cu128，CUDA 可用
主要依赖	opencv-python、torchvision、torchreid、gradio、Pillow、PyYAML、matplotlib、tqdm、scipy、pandas、seaborn 等
训练数据	yolov5\data\mydata，包含 9 张图片、9 个 XML 标注、9 个 YOLO txt 标签

3. 实验原理

3.1 YOLOv5 目标检测

YOLOv5 将目标检测任务看作单阶段回归问题，输入视频帧后直接输出目标类别、置信度和边界框。本实验使用 yolov5m.pt 作为主要检测模型，并在推理时筛选 person、car、motorcycle、bus、truck 等与作业相关的类别。

3.2 DeepSORT 目标追踪

DeepSORT 在检测框基础上为每个目标分配持续 ID。它综合卡尔曼滤波预测、检测框匹配和 ReID 外观特征，使同一辆车或同一个人在连续帧中保持相同 ID，为过线计数提供去重基础。

3.3 过线计数逻辑

原始代码只判断目标中心点是否位于计数线下方，可能导致目标第一次出现在线下方时被误计数。因此修改为跨线事件计数：同一 track ID 的上一帧中心点位于计数线上方，当前帧中心点移动到计数线下方时才计数。代码修改如下：修改 main.py 与 webui.py 中的 count_obj 函数，新增 track_history 记录上一帧中心点，并在 detect 开始时重置 count、count2、data 和 track_history，避免多次运行互相影响。

4. 实验步骤

4.1 环境搭建与依赖修复

1. 使用 Anaconda 的 torch 环境运行项目，验证 PyTorch GPU 可用，CUDA 设备为 RTX 4060 Laptop GPU。
2. 安装 opencv-python、PyYAML、matplotlib、tqdm、scipy、pandas、seaborn、easydict、yacs、gradio、torchreid、tensorboard 等依赖。
3. 针对当前 Python/PyTorch/Numpy/Pillow 版本，对旧项目源码进行兼容性修复。

问题	处理方式
torchreid 模块路径变化	将 torchreid.metrics 和 torchreid.utils 兼容到 torchreid.reid.metrics 与 torchreid.reid.utils。
PyTorch 2.6+ 权重加载默认值变化	在 torch.load 调用中加入 weights_only=False，用于加

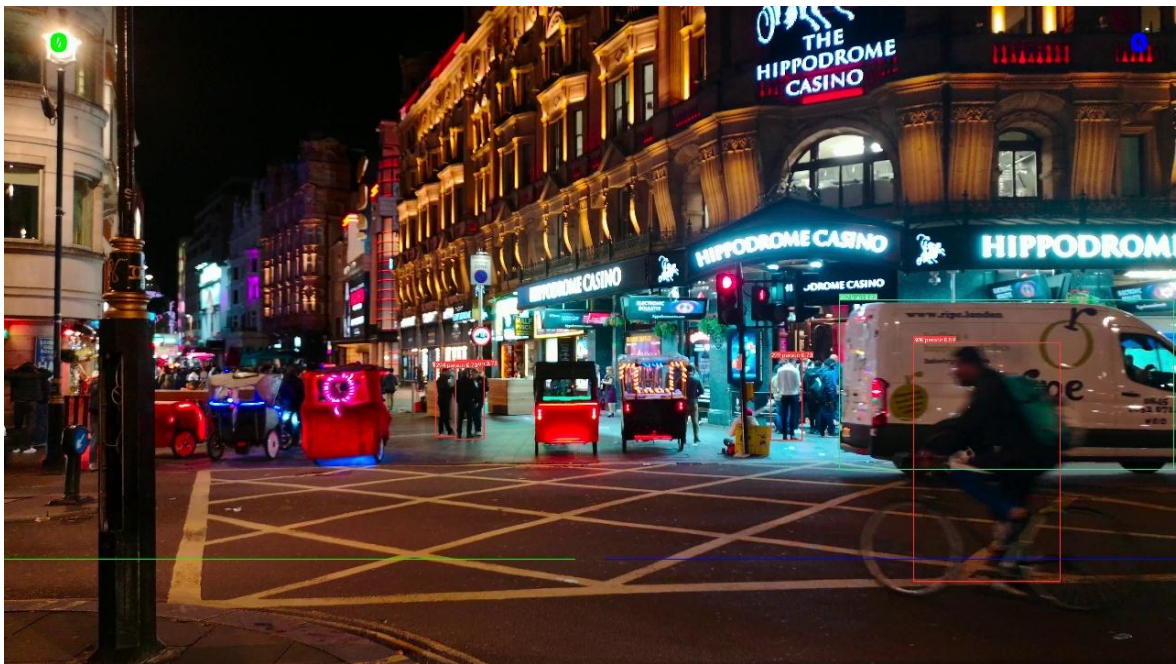
	载旧 YOLOv5 权重和训练权重。
NumPy 删除 np.float / np.int	替换为 Python 内置 float / int。
Pillow 删除 FreeTypeFont.getsize	改为优先使用 getbbox 计算文字宽高。
中文路径导致训练数据解析异常	将 mydata.yaml 改为 path: data/mydata 的相对路径写法，避免中文绝对路径乱码。

4.2 视频检测与追踪

首先使用 example.mp4 验证 pipeline，确认 YOLOv5 权重、DeepSORT 权重、OpenCV 视频读取与结果保存均可运行。随后使用正式作业视频进行检测追踪，主要命令如下：

```
python main.py --source C:\Users\XZBbb\Desktop\ai\课后作业视频\3007580-uhd_3840_2160_30fps.mp4 --yolo_model yolov5m.pt --device 0 --save-vid --save-txt --classes 0 2 3 5 7
```

输出项	路径或说明
处理后视频	C:\Users\XZBbb\Desktop\ai\相关源码和模型\VehicleCountor\runs\track\yolov5m_osnet_ibn_x1_0_M SMT17\3007580-uhd_3840_2160_30fps.mp4
轨迹文件	C:\Users\XZBbb\Desktop\ai\相关源码和模型\VehicleCountor\runs\track\yolov5m_osnet_ibn_x1_0_M SMT17\tracks\3007580-uhd_3840_2160_30fps.txt
轨迹记录格式	frame_id、track_id、bbox_left、bbox_top、bbox_width、bbox_height、占位字段、source_id



4.3 数据标注与格式转换

项目数据目录中包含 Pascal VOC XML 标注文件和 YOLO txt 标签。转换脚本为 `yolov5/data/mydata/voc2yolo_label.py`，类别配置为 `classes = ["car"]`。转换后每个标签行格式为 `class_id x_center y_center width height`，其中坐标为归一化值。

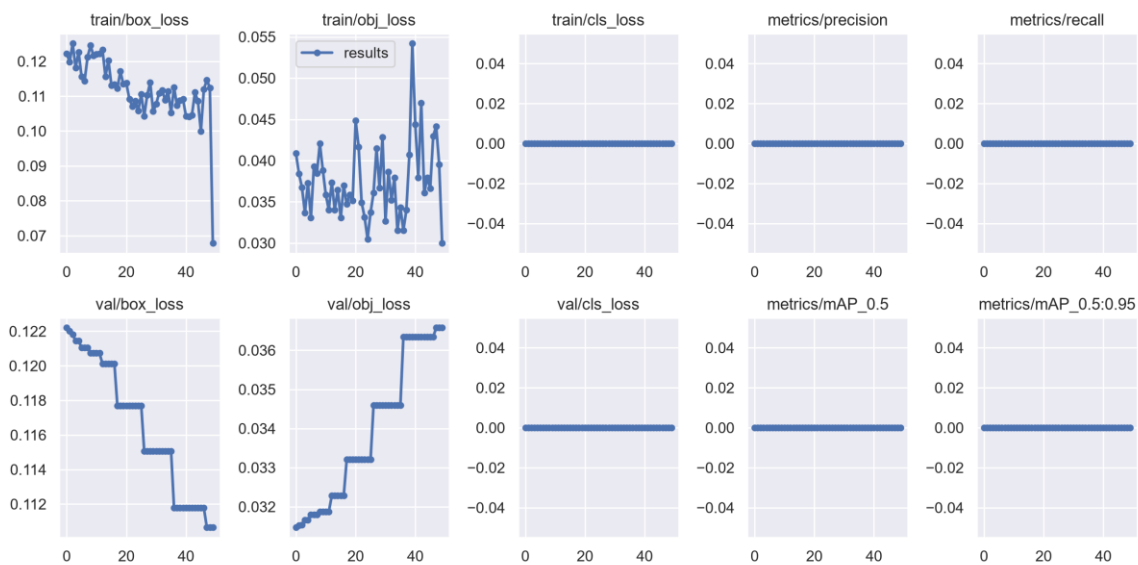
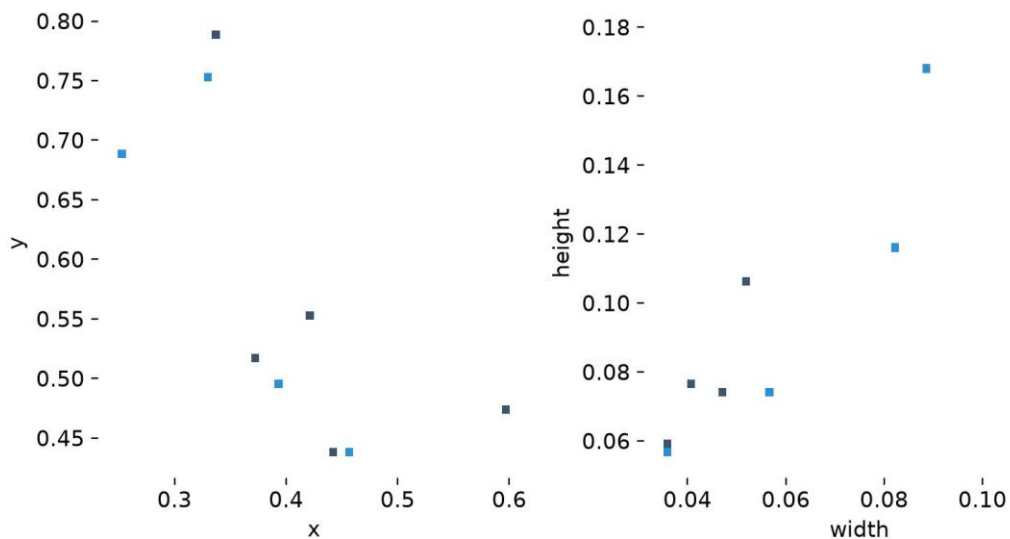
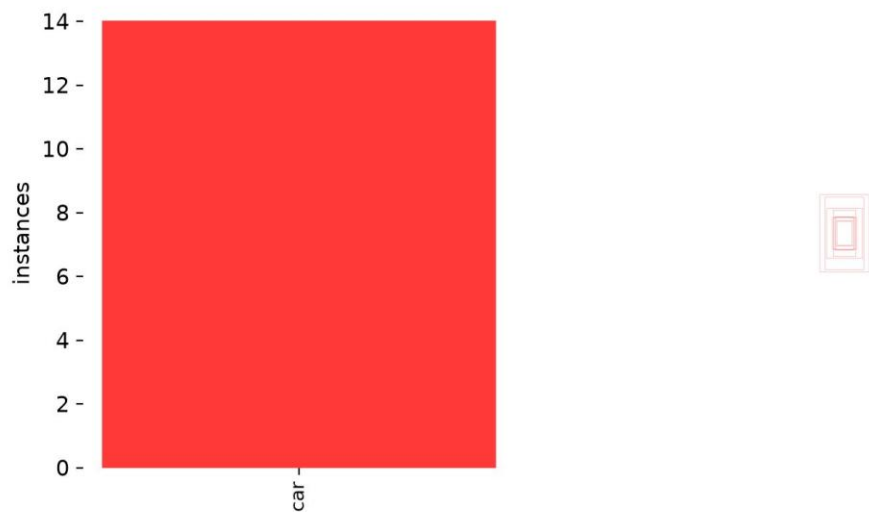
数据项	数量或状态
图片	9 张 jpg
VOC XML	9 个 xml
YOLO 标签	9 个 txt
数据划分	train、val、test 文本文件均存在
mydata.yaml	使用相对路径 path: data/mydata; nc: 1; names: ["car"]

4.4 模型训练

训练使用 YOLOv5m 架构和 `yolov5m.pt` 预训练权重，训练 50 个 epoch，batch size 为 2，输入尺寸为 640。训练命令如下：

```
python train.py --img 640 --batch 2 --epochs 50 --data data/mydata.yaml --  
weights ../yolov5m.pt --cfg models/yolov5m.yaml --device 0
```

参数	值
weights	/yolov5m.pt
cfg	models/yolov5m.yaml
data	data/mydata.yaml
epochs	50
batch size	2
img size	640
optimizer	SGD
save_dir	runs\train\exp5

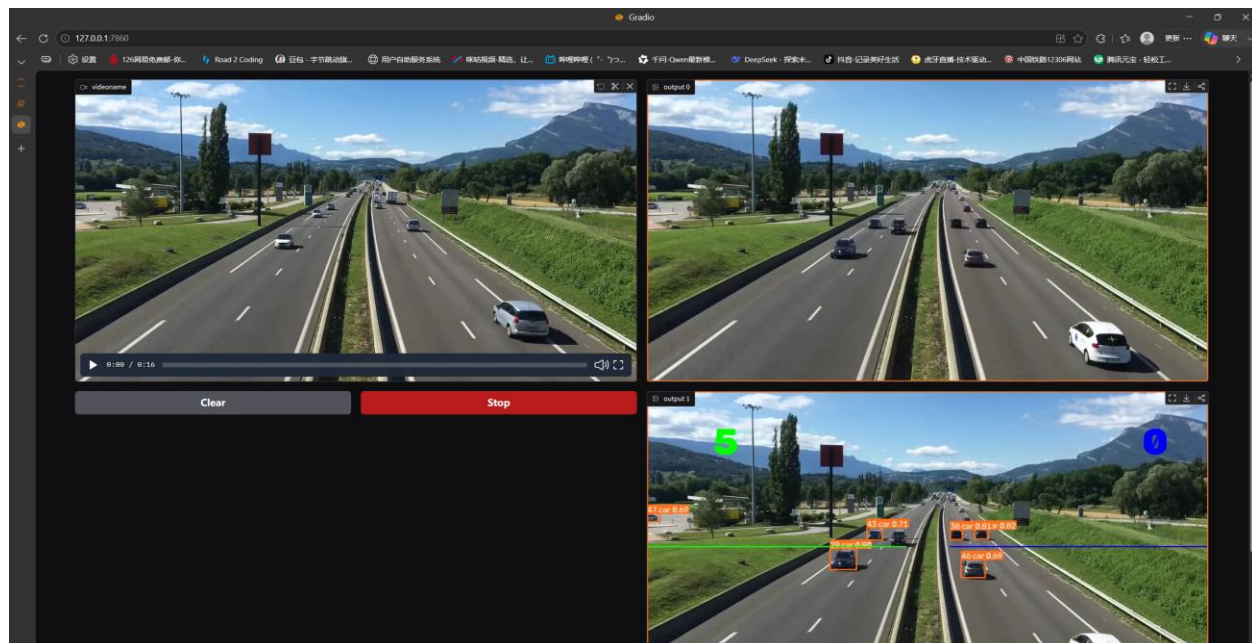


训练输出	结果
训练完成情况	50 epochs completed
best.pt	C:\Users\XZBbb\Desktop\ai\相关源码和模型 \VehicleCountor\yolov5\runs\train\exp5\weights\best.p t
last.pt	C:\Users\XZBbb\Desktop\ai\相关源码和模型 \VehicleCountor\yolov5\runs\train\exp5\weights\last.pt
权重大小	约 42.1 MB

4.5 Gradio 前端可视化

前端入口为 webui.py，使用 Gradio 提供视频上传页面。终端成功输出 Running on local URL: <http://127.0.0.1:7861>，说明前端服务已正常启动。前端用于展示原始视频帧与检测计数后的结果帧，主要用于验收前端可视化评分项。

- main.py 适合离线批量处理并保存正式 mp4 结果。
- webui.py 适合演示上传视频、实时显示检测框、目标 ID、计数线和计数数字。
- 正式 4K 作业视频已由 main.py 生成结果；前端可使用 example.mp4 做演示截图。



5. 实验结果与分析

实验最终完成了从环境搭建、依赖修复、视频检测、目标追踪、过线计数、数据转换、模型训练到前端演示的完整流程。正式作业视频生成了带检测框、目标 ID、计数线和计数数字的 mp4 文件，同时保存了 DeepSORT 轨迹 txt 文件。训练部分使用 YOLOv5m 完成 50 个 epoch，并生成 best.pt 与 last.pt。

评分点	完成情况
程序能否正常运行且测试视频能跑起来	已完成。example.mp4 和正式作业视频均已跑通。
神经网络模型训练程序能正常执行	已完成。YOLOv5m 训练 50 epoch，结果保存到 exp5。
数据标注程序与画框	已有 9 个 VOC XML 标注文件；可用 labellmg 进行演示与补充截图。
XML 转 txt 与训练	已完成。voc2yolo_label.py 可生成 labels/*.txt，mydata.yaml 已修正为相对路径。
Gradio 前端可视化	已完成。webui.py 成功启动本地 Gradio 服务。

结果说明：本次训练数据量较小，验证集中有效标签数为 0，因此训练指标不代表模型真实性能。正式视频检测结果主要依赖 yolov5m.pt 预训练能力和 DeepSORT 追踪能力。

6. 问题与解决方案

问题	原因	解决方案
torchreid.metrics / torchreid.utils 找不到	安装的 torchreid 包结构与项目旧代码不一致。	添加兼容导入，失败时改用 torchreid.reid.metrics / torchreid.reid.utils。
YOLO 权重加载失败	PyTorch 2.6 后 torch.load 默认 weights_only=True。	在 experimental.py、train.py、general.py 中指定 weights_only=False。

np.float / np.int 报错	NumPy 新版本移除了旧别名。	将相关代码改为 float 和 int。
Pillow getsize 报错	Pillow 新版 FreeTypeFont 不再提供 getsize。	使用 getbbox 计算文本宽高。
训练找不到数据路径	中文路径被命令行或 YAML 解析为乱码。	mydata.yaml 使用相对路径 path: data/mydata。
左右计数不准确	旧逻辑只判断当前位置在线下方，没有判断真实跨线。	新增 track_history，仅当目标从线上方跨到线下方时计数。

7. 实验总结

本实验完成了一个基于 YOLOv5 与 DeepSORT 的高速车流量计数系统。通过 YOLOv5m 进行目标检测，通过 DeepSORT 维护跨帧目标 ID，通过 OpenCV 绘制检测框、计数线和计数数字，并通过 Gradio 提供前端可视化入口。实验过程中还完成了数据标注格式转换和 YOLOv5m 50 epoch 训练，生成了可提交的权重文件、结果视频和轨迹文件。

从工程实践角度看，本实验的关键收获是：旧深度学习项目在新版本 Python、PyTorch、NumPy、Pillow 环境中运行时，常需要进行依赖兼容修复；同时，计数类任务不能只依赖目标当前位置，而应结合追踪 ID 和上一帧轨迹判断真实跨线事件。后续若要提升效果，应扩大标注数据集，修正验证集有效标签，并将左右区域计数进一步扩展为 person 与 vehicle 的类别计数。